

Annexe A

Documentation exhaustive des paramètres des solveurs implémentés

Cette annexe décrit en détail l'ensemble des paramètres configurables dans les différentes versions de CCAAnr[25].

A.1 Paramètres communs à toutes les versions (sauf MAB)

A.1.1 Paramètres d'exécution généraux

Paramètre	Description
<code>-inst</code>	Chemin vers le fichier CNF contenant l'instance à résoudre.
<code>-seed</code>	Graine aléatoire pour le générateur de nombres aléatoires. Valeur par défaut : 1.
<code>-ls_no_improv_steps</code>	Nombre de flips consécutifs sans amélioration du meilleur score avant de déclencher la diversification. Valeur par défaut : 200000.

A.1.2 Paramètres de pondération des clauses (smoothing)

Paramètre	Description
-swt_threshold	Seuil de $\bar{w}$ au-delà duquel le lissage des poids est déclenché. Valeur par défaut : 300 pour l'initialisation dynamique, 50 pour l'utilisateur.
-swt_p	Coefficient multiplicatif p dans la formule de lissage $w_c \leftarrow w_c \cdot p + \text{scale_ave}$. Valeur par défaut : 0,3.
-swt_q	Coefficient additif q utilisé pour calculer $\text{scale_ave} = (\text{threshold} + 1) * q$. Ajusté dynamiquement selon le ratio clauses/variables si non spécifié.

A.1.3 Paramètre d'aspiration

Paramètre	Description
-aspiration	Active (1) ou désactive (0) le mécanisme d'aspiration. L'aspiration recherche une variable avec score $> \bar{w}$ dans les clauses non satisfaites. Valeur par défaut : 0 (désactivée).

A.2 Paramètres spécifiques à la version bandit (MAB)

Les paramètres du bandit sont exposés comme options de ligne de commande : -mab_arms pour le nombre de bras échantillonnés, -mab_lambda pour le coefficient d'exploration, -mab_delay pour la profondeur de la récompense différée, et -mab_gamma pour le facteur d'actualisation.

Paramètre	Description
-mab_arms	Nombre de bras (clauses) échantillonnés aléatoirement pour l'évaluation de l'UCB à chaque étape. Valeur par défaut : 20.
-mab_lambda	Coefficient d'exploration λ dans la formule UCB : $\text{UCB}(c) = \frac{V_c}{n_c} + \lambda \sqrt{\frac{\ln N}{n_c + 1}}$. Valeur par défaut : 1,0.
-mab_delay	Profondeur de la mémoire pour la récompense différée. Définit le nombre d'étapes antérieures dont les clauses sont créditées lors d'une amélioration. Valeur par défaut : 20.
-mab_gamma	Facteur d'actualisation γ pour la récompense différée. La contribution d'une clause diminue de γ à chaque pas de distance. Valeur par défaut : 0,9.

A.3 Paramètres internes (non configurables en ligne de commande)

Ces paramètres sont définis dans le code et ne sont pas exposés comme options de ligne de commande dans la version étudiée.

Paramètre	Description
<code>max_tries</code>	Nombre maximal d'essais (redémarrages) autorisés. Valeur par défaut : 10000.
<code>max_flips</code>	Nombre maximal de flips par essai. Valeur par défaut : 20000000000 (effectivement infini).
<code>MAX_VARS</code>	Limite maximale du nombre de variables (compile-time). Valeur : 4 000 010.
<code>MAX_CLAUSES</code>	Limite maximale du nombre de clauses (compile-time). Valeur : 20 000 000.
<code>q_init</code>	Indicateur interne pour l'initialisation dynamique de q . Valeur par défaut : 0.
<code>ave_weight</code>	Poids moyen des clauses, mis à jour dynamiquement. Valeur initiale : 1.
<code>delta_total_weight</code>	Compteur d'incréments de poids cumulés pour déclencher le lissage.
<code>scale_ave</code>	Valeur calculée comme $(\text{threshold} + 1) * q_scale$, utilisée dans le lissage.

A.4 Paramètres des versions `CC_UNSAT` et `weight_conservation`

Ces versions n'introduisent pas de nouveaux paramètres configurables. Leur comportement est déterminé statiquement :

- **`CC_UNSAT`** : Le filtrage des clauses est toujours actif. Aucun paramètre ne permet de le désactiver dans cette version.
- **`weight_conservation`** : La conservation des poids est toujours active. Les poids ne sont réinitialisés qu'au premier essai (`current_try == 0`).

A.5 Valeurs par défaut récapitulatives

Paramètre	Valeur par défaut	Version concernée
seed	1	Toutes
ls_no_improv_times	200000	Toutes
p_scale	0,3	Toutes
q_scale	0,7 (ajustable)	Toutes
threshold	50	Toutes
aspiration	0 (désactivée)	Toutes
ArmNum_MAB	20	MAB
lambda_MAB	1,0	MAB
delay_MAB	20	MAB
gamma_MAB	0,9	MAB
max_tries	10000	Interne
max_flips	2000000000	Interne

Annexe B

Analyse détaillée du code source de CCAnr (implémentation C/C++)

Cette annexe présente une analyse approfondie du code source original de CCAnr[23] (version indiquée comme 1.1 sur le site sur lequel est déposé le code source original mais comme la version 2.0 [Version : 2018.01.28] dans le code source). L'analyse couvre les structures de données, les fonctions clés et les interactions entre les composantes de l'algorithme.

B.1 Structures de données principales (basis.h)

Le fichier `basis.h` définit l'ensemble des structures globales du solveur. La formule CNF est représentée par deux tableaux de littéraux : `var_lit[v]` pour la liste des occurrences de chaque variable, et `clause_lit[c]` pour la liste des littéraux de chaque clause. Chaque littéral est une structure contenant le numéro de clause, le numéro de variable et le sens (vrai ou faux). Le solveur maintient également un tableau `cur_soln` pour l'affectation courante, un tableau `score` pour le gain associé à chaque variable, et un tableau `clause_weight` pour la pondération des clauses.

La gestion des clauses non satisfaites repose sur deux structures de pile. La pile `unsat_stack` contient les indices des clauses actuellement non satisfaites, tandis que `unsatvar_stack` contient les variables apparaissant dans ces clauses, avec un compteur `unsat_app_count` pour chaque variable. Un tableau `conf_change` indique, pour chaque variable, si sa configuration a changé depuis son dernier flip – c'est le cœur du mécanisme de vérification de configuration. Enfin, la pile `goodvar_stack` stocke les variables ayant un score positif et une configuration changée, qui sont candidates prioritaires en mode glouton.

B.1.1 Macros de gestion des piles

Le code utilise deux macros pour la manipulation des piles :

```
1 #define pop(stack) stack[--stack ## _fill_pointer]
2 #define push(item, stack) stack[stack ## _fill_pointer++] = item
```

Listing B.1 – Macros push et pop

Ces macros permettent d'ajouter et de retirer des éléments de manière efficace.

B.2 Gestion des clauses non satisfaites (unsat et sat)

Les fonctions `unsat(clause)` et `sat(clause)` sont définies comme `inline` dans `cca.h` pour des raisons de performance.

B.2.1 Fonction `unsat(clause)`

La fonction `unsat(clause)` ajoute une clause à la pile `unsat_stack` et met à jour `unsat_app_count` pour chaque variable de la clause. Si une variable apparaît pour la première fois dans une clause non satisfaite, elle est ajoutée à `unsatvar_stack`.

```
1 inline void unsat(int clause)
2 {
3     index_in_unsat_stack[clause] = unsat_stack_fill_pointer;
4     push(clause, unsat_stack);
5
6     int v;
7     for(lit* p = clause_lit[clause]; (v = p->var_num) != 0; p++) {
8         unsat_app_count[v]++;
9         if(unsat_app_count[v] == 1) {
10             index_in_unsatvar_stack[v] = unsatvar_stack_fill_pointer;
11             push(v, unsatvar_stack);
12         }
13     }
14 }
```

Listing B.2 – Fonction `unsat(clause)`

B.2.2 Fonction sat(*clause*)

La fonction `sat(clause)` réalise l'opération inverse : elle retire la clause de la pile en remplaçant sa position par le dernier élément de la pile, et décrémente les compteurs de variables, en retirant celles qui n'apparaissent plus dans aucune clause non satisfaite.

```
1 inline void sat(int clause)
2 {
3     int index, last_unsat_clause;
4     last_unsat_clause = pop(unsat_stack);
5     index = index_in_unsat_stack[clause];
6     unsat_stack[index] = last_unsat_clause;
7     index_in_unsat_stack[last_unsat_clause] = index;
8
9     int v, last_unsat_var;
10    for(lit* p = clause_lit[clause]; (v = p->var_num) != 0; p++) {
11        unsat_app_count[v]--;
12        if(unsat_app_count[v] == 0) {
13            last_unsat_var = pop(unsatvar_stack);
14            index = index_in_unsatvar_stack[v];
15            unsatvar_stack[index] = last_unsat_var;
16            index_in_unsatvar_stack[last_unsat_var] = index;
17        }
18    }
19 }
```

Listing B.3 – Fonction sat

B.3 Initialisation init()

La fonction `init()` initialise l'affectation courante de manière aléatoire, puis calcule pour chaque clause le nombre de littéraux satisfaits (`sat_count`) et remplit `unsat_stack` en conséquence. Le score de chaque variable est calculé comme la somme des contributions des clauses non satisfaites auxquelles elle appartient. Une variable voit son score augmenter pour chaque clause non satisfaite qui la contient, et diminuer pour chaque *clause critique* (telle que définie en 2.5.1) dont elle est le littéral satisfait. Ce calcul incrémental est essentiel pour la performance. La pile `goodvar_stack` est ensuite remplie avec les variables ayant un score strictement positif.

B.4 Opération de flip(flipvar)

La fonction `flip(flipvar)` est la plus délicate du solveur. Elle inverse la valeur de la variable choisie et met à jour l'ensemble des structures en conséquence. Pour chaque clause contenant `flipvar`, quatre cas sont possibles selon l'état antérieur de la clause et le sens du littéral. Si la clause devient satisfaite (passage de 0 à 1 littéral vrai), elle est retirée de `unsat_stack` et les scores de toutes ses variables sont décrémentés du poids de la clause. Si la clause devient non satisfaite (passage de 1 à 0), elle est ajoutée à `unsat_stack` et les scores de toutes ses variables sont incrémentés. Si la clause passe de 1 à 2 littéraux vrais ou de 2 à 1, seules les variables impliquées voient leur score modifié. Après la mise à jour des clauses, la fonction met à jour `conf_change` : la variable flippée voit son `conf_change` passer à 0, tandis que tous ses voisins passent à 1. Enfin, la pile `goodvar_stack` est réévaluée.

```
1 void flip(int flipvar)
2 {
3     cur_soln[flipvar] = 1 - cur_soln[flipvar];
4     // ...
5     int org_flipvar_score = score[flipvar];
6
7     // update related clauses and neighbor vars
8     for(lit *q = var_lit[flipvar]; (c = q->clause_num) >= 0; q++) {
9         // ...
10    }
11
12    score[flipvar] = -org_flipvar_score;
13    int index;
14
15    conf_change[flipvar] = 0;
16
17    // ...
18
19    int* p;
20    for(p = var_neighbor[flipvar]; (v = *p) != 0; p++) {
21        conf_change[v] = 1;
22
23        if(score[v]>0 && already_in_goodvar_stack[v] ==0) {
24            push(v,goodvar_stack);
25            already_in_goodvar_stack[v] = 1;
26        }
27    }
28 }
```

Listing B.4 – Extrait de la fonction flip (flipvar)

B.5 Pondération des clauses (cw.h)

La pondération des clauses suit un schéma additif avec lissage périodique. La fonction `update_clause_weights()` incrémente le poids de chaque clause non satisfaite et met à jour les scores en conséquence. Un compteur `delta_total_weight` suit l'augmentation cumulée ; lorsqu'il dépasse le nombre de clauses, `ave_weight` est incrémenté et, si celui-ci dépasse un seuil, un lissage est déclenché. La fonction `smooth_clause_weights()` applique la formule $w_c \leftarrow w_c \cdot p + \text{scale_ave}$ puis recalcule les scores. Les paramètres p et q (dont dépend `scale_ave`) sont ajustés dynamiquement selon le ratio clauses/variables.

```
1 void update_clause_weights()
2 {
3     int i,v;
4
5     for(i=0; i < unsat_stack_fill_pointer; ++i)
6         clause_weight[unsat_stack[i]]++;
7
8     for(i=0; i<unsatvar_stack_fill_pointer; ++i)
9     {
10         v = unsatvar_stack[i];
11         score[v] += unsat_app_count[v];
12         if(score[v]>0 && conf_change[v]==1 && already_in_goodvar_stack[v]
13             ==0)
14         {
15             push(v,goodvar_stack);
16             already_in_goodvar_stack[v] =1;
17         }
18     }
19
20     delta_total_weight+=unsat_stack_fill_pointer;
21     if(delta_total_weight>=num_clauses)
22     {
23         ave_weight+=1;
24         delta_total_weight -= num_clauses;
25
26         //smooth weights
27         if(ave_weight>threshold)
28             smooth_clause_weights();
29     }
```

Listing B.5 – Fonction `update_clause_weights()`

B.6 Prétraitement (preprocessor.h)

Le prétraitement effectue une propagation unitaire (unit propagation) : les clauses unitaires sont traitées itérativement, en fixant la variable concernée et en supprimant les clauses satisfaites ou en réduisant celles qui deviennent unitaires. Ce prétraitement simplifie la formule et peut réduire considérablement sa taille.

Annexe C

Environnement d'exécution des tests et détails techniques

Cette annexe présente l'environnement logiciel utilisé pour l'exécution des tests expérimentaux.

C.1 Environnement de test

C.1.1 Infrastructure matérielle

Les expériences ont été menées sur le cluster de calcul de l'équipe COALA au LIS. Huit machines du cluster ont été utilisées, référencées sous les noms `lis-cluster-coala-49` à `lis-cluster-coala-64`. Chaque machine possède la configuration suivante :

- Rack : Precision DELL 7920
- Processeur : 2 processeurs Intel Xeon Gold 5218R à 2,1 GHz (64 bits, 20 cœurs chacun)
- Mémoire cache : 27,5 Mo
- Mémoire vive : 192 Go
- Carte graphique : NVIDIA T400 4 Go (384 cœurs CUDA) – non utilisée pour ces tests
- Système d'exploitation : Linux Ubuntu LTS 22.04.2
- Compilateur : GCC 11.4.0
- Interpréteur Python : 3.10.12
- Plage d'adresses IP : 192.168.200.58 à 192.168.200.71

C.1.2 Environnement logiciel

L’environnement logiciel sur lequel ont été menées les expérimentations est le suivant :

- Système d’exploitation : Linux Ubuntu LTS 22.04.2
- Compilateur : GCC 11.4.0 (pour les tests sur cluster) et GCC/G++ 14.2.0 (pour la compilation locale)
- Interpréteur Python : 3.10.12 (pour les scripts d’analyse des résultats)
- Gestionnaire de versions : Git

C.2 Compilation Scripts de lancement

Toutes les versions ont été compilées sur une machine personnelle avec le compilateur GCC/G++ 14.2.0 et les options de compilation suivantes :

```
1 g++ cca.cpp -O3 -static -o <nom_executable>
```

Listing C.1 – Options de compilation

Les exécutables déjà compilés, les scripts de lancement des tests ainsi que les fichiers d’instances ont été transférés sur les serveurs du cluster avant l’exécution.

Les scripts de lancement des tests ont été écrits en Bash. Chaque script exécute séquentiellement les instances sur une machine donnée, avec un timeout de 3600 secondes par instance.

C.3 Recueil et format des résultats

À la fin de chaque salve, sur chaque machine utilisée, un fichier CSV est créé dans le dossier RESULTS. Le nom du fichier suit le format :

```
{nom_de_l_executable}_{nom_du_dossier_des_instances}_result.csv
```

Format CSV pour les versions refactorisée, CC_UNSAT et weight_conservation

Pour ces trois versions (qui partagent la même structure de paramètres), le fichier CSV contient une ligne par instance avec les colonnes suivantes :

Colonne	Description
algorithm_version	Nom de la version (ex : refactor, cc_unsat, weight_conservation)
instance	Nom du fichier de l'instance SAT
nb_vars	Nombre de variables de l'instance
nb_clauses	Nombre de clauses de l'instance
ratio	Rapport clauses/variables
nb_literals	Nombre total de littéraux dans la formule
avg_min_max	Statistiques sur la longueur des clauses (moyenne, min, max)
seed	Graine aléatoire utilisée
ls_no_improv_times	Nombre de flips sans amélioration avant diversification
p_scale	Coefficient de lissage multiplicatif des poids
q_scale	Coefficient de lissage additif des poids
threshold	Seuil déclenchant le lissage des poids
scale_ave	Valeur d'échelle moyenne calculée
aspiration	Booléen indiquant si l'aspiration est activée
sat	Résultat : "SAT" si une solution est trouvée, "UNKNOWN" sinon, "NULL" si timeout
steps	Nombre de flips effectués, "NULL" si timeout
resolve_time	Temps de résolution en secondes, "NULL" si timeout

Format CSV pour la version bandit (MAB)

La version bandit comporte des paramètres supplémentaires liés à l'algorithme UCB. Le fichier CSV contient donc des colonnes additionnelles :

Colonne	Description
algorithm_version	Version "mab"
instance	Nom du fichier de l'instance SAT
nb_vars	Nombre de variables de l'instance
nb_clauses	Nombre de clauses de l'instance
ratio	Rapport clauses/variables
nb_literals	Nombre total de littéraux dans la formule
avg_min_max	Statistiques sur la longueur des clauses (moyenne, min, max)
seed	Graine aléatoire utilisée
ls_no_improv_times	Nombre de flips sans amélioration avant diversification
p_scale	Coefficient de lissage multiplicatif des poids
q_scale	Coefficient de lissage additif des poids
threshold	Seuil déclenchant le lissage des poids
scale_ave	Valeur d'échelle moyenne calculée
aspiration	Booléen indiquant si l'aspiration est activée
nb_arms_MAB	Nombre de bras (clauses) échantillonnés pour la sélection UCB
lambda_MAB	Coefficient d'exploration pour l'UCB
delay_MAB	Profondeur de la récompense différée
gamma_MAB	Facteur d'actualisation pour la récompense différée
sat	Résultat : "SAT" si une solution est trouvée, "UNKNOWN" sinon, "NULL" si timeout
steps	Nombre de flips effectués, "NULL" si timeout
resolve_time	Temps de résolution en secondes, "NULL" si timeout

Interprétation des valeurs spéciales

Deux cas particuliers sont distingués pour la colonne **sat** :

- La valeur NULL indique que le solveur n'a pas pu consommer tous ses essais (redémarrages) dans le temps imparti. La recherche a été interrompue par le timeout avant d'avoir épuisé toutes les tentatives.
- La valeur UNKNOWN indique que le solveur a consommé tous ses essais (tous les redémarrages) mais n'a pas trouvé de solution. La recherche s'est donc achevée normalement sans succès.

Annexe D

Fusion des contributions

Cette annexe présente la structure du code unifié de CCAnr, qui intègre l'ensemble des quatre contributions développées dans ce stage. Contrairement aux versions séparées où chaque contribution est compilée dans un exécutable distinct, cette version permet d'activer ou de désactiver chaque fonctionnalité indépendamment via des arguments de ligne de commande, sans avoir à recompiler le solveur.

D.1 Options de ligne de commande

TABLE D.1 – Options de ligne de commande pour le code unifié

Option	Description
-inst	Instance CNF à résoudre (obligatoire)
-seed	Graine aléatoire (défaut : 1)
-aspiration	Active le mécanisme d'aspiration
-swt_threshold	Seuil de lissage des poids (défaut : 50)
-swt_p	Coefficient multiplicatif de lissage (défaut : 0.3)
-swt_q	Coefficient additif de lissage (défaut : 0.7)
-ls_no_improv_steps	Nombre de flips sans amélioration (défaut : 200000)
-mab	Active le bandit UCB
-mab_arms	Nombre de bras échantillonnés (défaut : 20)
-mab_lambda	Coefficient d'exploration (défaut : 1.0)
-mab_delay	Profondeur de la récompense différée (défaut : 20)
-mab_gamma	Facteur d'actualisation (défaut : 0.9)
-cc_unsat	Active le filtrage des clauses CC UNSAT
-weight_conservation	Active la conservation des poids entre redémarrages

D.2 Exemples d'utilisation

```
1 # Version originale (aucune contribution)
2 ./CCAnr_general -inst instance.cnf
3
4 # Bandit UCB
5 ./CCAnr_general -inst instance.cnf -mab -mab_arms 30 -mab_lambda 1.5
6
7 # Conservation des poids
8 ./CCAnr_general -inst instance.cnf -weight_conservation
9
10 # Filtrage CC UNSAT
11 ./CCAnr_general -inst instance.cnf -cc_unsat
12
13 # Bandit + conservation des poids
14 ./CCAnr_general -inst instance.cnf -mab -weight_conservation
15
16 # Toutes les options compatibles
17 ./CCAnr_general -inst instance.cnf -seed 42 -aspiration -mab -
    weight_conservation
```

Listing D.1 – Exemples de lancement

D.3 Incompatibilités

Les options `-mab` et `-cc_unsat` sont mutuellement exclusives et ne peuvent pas être activées simultanément.

Bibliographie

- [1] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2), 235-256.
- [2] Biere, A., Heule, M., & van Maaren, H. (2021). *Handbook of Satisfiability : Second Edition*. IOS Press.
- [3] Cai, S., & Su, K. (2011). Local search with configuration checking for SAT. In *2011 IEEE 23rd International Conference on Tools with Artificial Intelligence* (pp. 59-66). IEEE.
- [4] Cai, S., & Su, K. (2012). Configuration checking with aspiration in local search for SAT. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence* (pp. 434-440).
- [5] Cai, S., Su, K., Local search for Boolean Satisfiability with configuration checking and subscore, *Artificial Intelligence* 204 (2013) 75–98.
- [6] Cai, S., Luo, C., & Su, K. (2015). CCAnr : A configuration checking based local search solver for non-random satisfiability. In *SAT 2015* (pp. 1-8). Springer.
- [7] Cai, S., Su, K., & Sattar, A. (2011). Local search with edge weighting and configuration checking heuristics for minimum vertex cover. *Artificial Intelligence*, 175(9-10), 1672-1696.
- [8] Cherif, M. S., Habet, D., & Terrioux, C. (2021). *Combining VSIDS and CHB Using Restarts in SAT*. In Proceedings of the 27th International Conference on Principles and Practice of Constraint Programming (CP 2021), LIPIcs, Vol. 210, pp. 20 :1–20 :19.
- [9] Cook, S. A. (1971). The complexity of theorem-proving procedures. In *Proceedings of the third annual ACM symposium on Theory of computing* (pp. 151-158).
- [10] Davis, M., & Putnam, H. (1960). A computing procedure for quantification theory. *Journal of the ACM*, 7(3), 201-215.
- [11] Davis, M., Logemann, G., & Loveland, D. (1962). A machine program for theorem-proving. *Communications of the ACM*, 5(7), 394-397.
- [12] Glover, F. (1989). Tabu search - Part I. *ORSA Journal on Computing*, 1(3), 190-206.
- [13] Glover, F. (1990). Tabu search - Part II. *ORSA Journal on Computing*, 2(1), 4-32.
- [14] Hoos, H. H. (2002). An adaptive noise mechanism for WalkSAT. In *AAAI/IAAI* (pp. 655-660).
- [15] Hoos, H. H., & Stutzle, T. (2005). *Stochastic local search : Foundations and applications*. Elsevier.
- [16] Hutter, F., Tompkins, D. A. D., & Hoos, H. H. (2002). Scaling and probabilistic smoothing : Efficient dynamic local search for SAT. In *CP 2002* (pp. 233-248). Springer.

- [17] Li, C. M., & Huang, W. Q. (2005). Diversification and determinism in local search for satisfiability. In *SAT 2005* (pp. 158-172). Springer.
- [18] McAllester, D., Selman, B., & Kautz, H. (1997). Evidence for invariants in local search. In *AAAI/IAAI* (pp. 321-326).
- [19] Selman, B., Levesque, H. J., & Mitchell, D. G. (1992). A new method for solving hard satisfiability problems. In *AAAI-92* (pp. 440-446).
- [20] Selman, B., Kautz, H. A., & Cohen, B. (1994). Noise strategies for improving local search. In *AAAI-94* (pp. 337-343).
- [21] Thornton, J., Pham, D. N., Bain, S., & Ferreira Jr, V. (2004). Additive versus multiplicative clause weighting for SAT. In *AAAI-2004* (pp. 191-196).
- [22] Zheng, J., et al. (2022). BandMaxSAT : A local search MaxSAT solver with multi-armed bandit. In *IJCAI-22*.
- [23] CCAnr source code. Available online at : <http://lcs.ios.ac.cn/~caisw/SAT.html>
- [24] Kissat_MAB source code. Available online at : <https://github.com/Simewu/top-SAT-solvers-2021>
- [25] Contributions source code. Available online at : https://gitlab.lis-lab.fr/lois.khirdidine/stage_sat_m2